



Security Assessment Report



DepositWallet

March 2026

Prepared for Polymarket

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	4
Security Overview	5
System-Level Model	5
Trust Boundaries and Security Assumptions	6
Role Hierarchy	6
Key Trust Assumptions	7
Attack Vectors Considered	7
Access Control	7
Value Flows and Economic Risks	8
Dependency Surfaces	8
Cross-Protocol Interactions	9
Timing and Ordering	9
Verification Checks	9
Assembly and Low-Level Safety	9
Signature and EIP-712 Integrity	10
Access Control Completeness	10
Reentrancy and Execution Control	11
Settlement Integrity	11
Fund Extraction Safety	11
Inherited Contract Correctness	12
Findings Summary	13
Severity Matrix	13
Detailed Findings	14
Low Severity Issues	15
L-01: Single reverting batch in proxy() grieves all other users/wallets in the transaction	15
Informational Severity Issues	19
I-01: Missing on-chain wallet authenticity check in proxy()	19
I-02: Timelock delay has no upper bound	20
Disclaimer	21
About Certora	21

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
DepositWallet	Polymarket/deposit-wallet	2d1ca79 (initial commit) a22a420 (fix commit)	Solidity

Project Overview

This document describes the security review of **Polymarket DepositWallet**. The work was undertaken from **March 4th, 2026** to **March 11th, 2026**.

The following contract list is included in our scope:

`deposit-wallet/src/*`

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

DepositWallet is a minimal, upgradeable smart-contract wallet system for managing ERC-20 and ERC-1155 token operations through signature-based batch execution.

Wallets are deployed as deterministic ERC-1967 UUPS proxies by a factory contract. Each wallet has a single owner who authorizes batches of arbitrary calls via signatures. Owners can delegate signing authority to time-limited session signers. The primary intent behind the deposit wallet is to custody user funds with a timelock in order to remove vectors where a user invalidates their order after off-chain settlement.

Assessment Methodology

Our assessment approach combines design-level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design-level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context-aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow-up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Security Overview

System-Level Model

The Polymarket DepositWallet is a minimal, upgradeable smart-contract wallet system designed to custody user funds on Polygon for Polymarket's prediction market. The system's core purpose is to prevent users from invalidating off-chain-matched orders between matching and on-chain settlement. There should be no action a user can take via DepositWallet that causes on-chain settlement initiated by Polymarket to fail.

The architecture consists of two primary contracts:

- **DepositWalletFactory** — Deploys deterministic ERC-1967 UUPS proxy wallets via Solady's LibClone. Manages a three-tier role hierarchy (Factory Owner → Admin → Operator). The factory owner maintains a whitelist of authorized implementations — only whitelisted implementations can be used for new wallet deployments or UUPS upgrades. Also stores the global `timelockDelay` parameter.
- **DepositWallet** — The core wallet implementation. Owners or time-limited session signers authorize batches of arbitrary calls via EIP-712 signatures. The wallet validates these signatures through ERC-1271, which external dApps like Polymarket's exchange also use for on-chain settlement. Session signers use an ERC-6492 envelope format and are restricted from calling the wallet itself. Ownership transfers follow a two-step process with EIP-712 signature confirmation. An owner-controlled pause mechanism with a configurable timelock enables asset withdrawals.

Supporting libraries include a custom Ownable with EIP-712 signature-based handover, WalletLib for EIP-712 hashing, and SessionSignerLib for session signer detection.

The system relies heavily on Solady for cryptographic primitives (ECDSA, EIP712), proxy infrastructure (UUPSUpgradeable, LibClone, Initializable), access control (OwnableRoles), token handling (SafeTransferLib, ERC1155), and receiver functionality (Receiver, ERC1271).

Trust Boundaries and Security Assumptions

Role Hierarchy

Role	Trust Level	Boundary	Capabilities
Factory Owner	Trusted	Highest privilege	Add/remove admins, authorize/unauthorize implementations, set default implementation, upgrade factory (UUPS)
Admin (ROLE_0)	Trusted	Factory configuration	Add/remove operators, set timelockDelay for all wallets
Operator (ROLE_1)	Semi-trusted	Relayer/settlement	Deploy wallets (controls _owners array), proxy signed batches to wallets
Wallet Owner	Semi-trusted	User-controlled key	Sign batches, pause wallet, withdraw after timelock, transfer ownership, upgrade wallet (only to factory-authorized implementations), manage session signers
Session Signer	Semi-trusted	Time-limited delegate	Sign batches only (cannot call wallet itself - target != address(this) restriction)
Anyone	Untrusted	Permissionless	Complete ownership handover (requires valid EIP-712 signature from pending owner)

Key Trust Assumptions

1. **Factory address is immutable:** Stored as an ERC-1967 immutable arg in the proxy bytecode, decoded at runtime. Cannot be changed after deployment. This anchors the `onlyFactory` modifier.
2. **Solady implementations are correct:** The system delegates all cryptographic operations (ECDSA recovery, EIP-712 hashing), proxy mechanics, and token transfers to Solady's audited library.
3. **Operators cannot forge signatures:** The operator can submit any batch to any wallet, but the wallet validates EIP-712 signatures at the wallet level. The operator is a relay, not an authenticator.
4. **Off-chain matching checks state at match time:** Polymarket's off-chain system checks allowances, balances, pause status, and signature validity when matching. The timelock duration exceeds the matching-to-settlement window.
5. **Pausing cancels all limit orders:** A user pausing their wallet (or triggering an ownership change) causes the off-chain system to cancel all outstanding orders, preventing stale batch execution.
6. **Collateral token is trusted:** Wallet funds will be denominated in Polymarket's own `CollateralToken` (a vault wrapper). This mitigates external token risks such as blacklisting, pausing, or fee-on-transfer behavior. The `CollateralToken` is planned to ship before `DepositWallet` goes live.

Attack Vectors Considered

Access Control

- **Operator privilege abuse** — Operator controls the `_owners` array in `deploy()` — can deploy wallets with arbitrary or zero-address owners. Wallet-level signature validation prevents operators from executing unauthorized batches.
- **Session signer escalation** — The `target != address(this)` restriction prevents session signers from calling `onlySelf` functions (`authorizeSessionSigner`, `transferOwnership`, `upgrade`) via self-call. Paused-only functions (`pause`, `withdrawERC20`, etc.) have a separate `onlyOwner` check that would reject self-calls regardless, since `msg.sender` during a self-call is the wallet address, not the EOA owner. No `delegatecall` path exists in the current code. Session signers can direct calls to the

factory address, but the factory's state-changing functions have their own role-based access control (`onlyAdmin/onlyOperator`), so such calls would revert.

- **Permissionless ownership handover** — `completeOwnershipHandover()` is callable by anyone but relies on EIP-712 signature verification against the `pendingOwner`.
- **Factory owner as centralization point** — The factory owner controls the implementation whitelist. However, wallet upgrades require an owner-signed batch — the factory owner can authorize a malicious implementation, but cannot force any wallet to adopt it. The mitigation is that each wallet owner must explicitly sign an upgrade to a new implementation.

Value Flows and Economic Risks

- **Arbitrary call execution** — `execute()` performs low-level `.call{value:}(data)` to any target with any calldata. On-chain, this is gated by the `onlyFactory` modifier (only the factory can call `execute`), the `onlyOperator` role (only operators can call `factory.proxy()`), EIP-712 signature validation, nonce-based replay protection, and deadline enforcement. The team confirmed that Polymarket's off-chain system also performs validation and filtering of batch contents before operator submission, providing a defense-in-depth layer.

Dependency Surfaces

- **Solady library** — 10 Solady modules used (`UUPSUpgradeable`, `OwnableRoles`, `Initializable`, `ERC1271`, `EIP712`, `ECDSA`, `LibClone`, `SafeTransferLib`, `Receiver`, `ERC1155`). A vulnerability in any of these directly impacts the system.
- **Memory slot coupling** — `isValidSignature()` stores the resolved signer at fixed memory slot `0x80` and advances the free memory pointer to `0xa0`. The subsequent `super.isValidSignature()` call (Solady's `ERC1271`) must not overwrite this slot. This coupling is fragile across Solady upgrades but verified correct with the current version.
- **Cross-contract state coupling** — The `onlyPaused` modifier reads `timelockDelay` from the factory via an external call at check time. The factory address is immutable, but the `timelockDelay` value is mutable by the admin and applies retroactively to all wallets — including those already paused.

Cross-Protocol Interactions

- **Polymarket Exchange integration** — The exchange contract calls `isValidSignature()` on each `DepositWallet` during on-chain settlement (ERC-1271 validation) and pulls funds via ERC-20 allowances. The `_erc1271CallerIsSafe()` override returns `true` when `msg.sender == factory()`, using a simplified verification path for factory-originated calls.
- **Session signer for trading** — Session signers enable time-limited delegation for automated trading. The ERC-6492 envelope format uses magic bytes for detection.

Timing and Ordering

- **Timelock consistency** — Session signer expiry uses strict `<` (invalid at exact expiry). Batch deadline uses `<=` (valid at exact deadline). Ownership handover deadline uses `<=`. Timelock check uses strict `>` (must be strictly after delay).
- **Re-pausability** — `pause()` can be called when already paused, overwriting the timestamp and resetting the timelock countdown. This is a self-grief vector for the owner.
- **Batch execution during pause** — The `execute()` function has no pause check — pausing only gates owner-direct functions. Pre-signed batches can still be submitted by operators during the timelock window.

Verification Checks

Assembly and Low-Level Safety

- **Memory slot signer passing** — Verified that the `mstore` of the resolved signer at slot `0x80` in `isValidSignature()` does not overwrite a to-be-used memory slot, and that the slot is not overwritten before `_erc1271Signer()` reads it back. The free memory pointer is advanced to `0xa0` before the super call, ensuring subsequent allocations do not clobber the signer. FMP is never left stale.
- **Assembly overflow infeasibility** — The nonce increment (`add(executedNonce, 1)`) uses unchecked assembly, but overflow requires 2^{256} batch executions — computationally infeasible. The pause timestamp is written directly from `block.timestamp` — multiple `pause()` calls in the same block produce the same value (no stacking), so overflow via repeated pausing is impossible.

Signature and EIP-712 Integrity

- **Unhappy signature paths** — Verified that approving `address(0)` as a session signer has no practical effect (detection returns `address(0)` for non-session-signer signatures). Verified that providing an invalid signature for ownership handover with `address(0)` as pending owner is blocked (`pendingOwner != address(0)` check precedes signature verification). `ECDSA.recoverCalldata` returns `address(0)` for invalid signatures, and the `require(recovered == pending)` check rejects this. Verified that signatures abusing the ERC-6492 magic bytes suffix would be routed through session signer validation, where the extracted address must be authorized — a false positive in detection does not bypass authentication.
- **Batch digest uniqueness** — Different batch contents always produce different EIP-712 digests. The Batch struct hash includes `wallet`, `nonce`, `deadline`, and a hash of all `Call` structs (each encoding `target`, `value`, `keccak256(data)`). No way to vary struct fields and produce the same hash, excluding computationally infeasible collisions in the bytes32 space.
- **Overall signature scheme correctness** — EIP-712 domain separation, typehash construction, and ECDSA verification work correctly end-to-end with no unintended collisions.

Access Control Completeness

- **Session signer cannot reach onlySelf functions** — The `target != address(this)` check is the sole defense for `onlySelf` functions. Verified no alternative path exists (no `delegatecall`, no indirect routing).
- **Wallet owner cannot self-upgrade** — Wallet upgrades require a batch execution path (owner signs → operator submits via `factory.proxy()` → wallet self-calls `upgrade()` → `_authorizeUpgrade` checks `factory whitelist`). The owner cannot call `upgradeToAndCall` directly on the proxy because `_authorizeUpgrade` has `onlySelf`, which rejects external callers.
- **Withdraw/revoke token calls cannot reach onlySelf** — The `_token` parameter in `withdrawERC20()`, `revokeAllowance()`, etc. is used as the target of a token call (e.g., `_token.safeTransfer()`). Setting `_token` to the wallet's own address would call into the wallet, but `safeTransfer` and `safeApprove` use specific function selectors that

don't match any `onlySelf` function — and these functions are gated by `onlyPaused` + `onlyOwner`, limiting the caller to a trusted role.

- **All functions have proper access controls** — Every state-changing function has the expected modifier (`onlyFactory`, `onlySelf`, `onlyOwner`, `onlyPaused`, `onlyAdmin`, `onlyOperator`, or `onlyOwner` on `factory`). All happy paths work as expected.

Reentrancy and Execution Control

- **Batch execution reentrancy** — Verified that users batching calls which give them execution control (e.g., calling a contract that calls back) cannot bypass protections. The nonce is incremented before any calls execute, preventing same-batch replay. Cross-batch reentrancy (re-entering `factory.proxy()` with the next nonce) requires the `onlyOperator` role, which external contracts do not have.
- **ERC-1155 withdrawal callback** — `withdrawERC1155()` calls `safeBatchTransferFrom`, which invokes `onERC1155BatchReceived` on the recipient if it is a contract. Verified this cannot be abused via reentrancy — the function is `onlyPaused` + `onlyOwner`, no wallet state is modified after the external call, and the caller (owner) controls the `_to` address.

Settlement Integrity

- **No bypass of settlement protections** — Verified that all actions which could cause on-chain settlement to revert (revoking allowances, moving funds, pausing) are properly guarded. Allowance revocation and fund withdrawal require `onlyPaused` + `onlyOwner` with elapsed timelock. `pause()` requires `onlyOwner` and triggers off-chain order cancellation. Batch execution is the only path for programmatic operations, gated by `onlyFactory` + `onlyOperator` + signature validation.

Fund Extraction Safety

- **No unintended withdrawal paths** — Verified that the exposed `isValidSignature()` cannot be abused to allow a collateral token contract to call into the wallet in a way that extracts funds or gives approvals via ERC1271 permit routing. Verified that arbitrary calls in batch execution cannot exploit special token permissions (e.g., bridge contracts with `bridgeOnly` burn rights that bypass approvals) — such vectors depend on external

protocol design and are mitigated by off-chain batch filtering as confirmed by the Polymarket team, and furthermore no such periphery contract exists.

- **Gas griefing via arbitrary calls** — Users could sign gas-heavy arbitrary calls to drain the operator's gas tokens. This is not protected on-chain, but off-chain protections (e.g., cap on the gas provided to transactions) are in place as confirmed by the Polymarket team.

Inherited Contract Correctness

- **Solady contracts used correctly** — Verified that inherited Solady contracts (UUPSUpgradeable, OwnableRoles, Initializable, ERC1271, EIP712, Receiver) are integrated correctly — storage slots do not collide with custom slots, override hooks behave as expected, owner initialized as required by inherited contract, `_disableInitializers()` is called in both constructors and others.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	1	1	-
Informational	2	2	1
Total	3	3	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Single reverting batch in proxy() grieves all other users/wallets in the transaction	Low	Acknowledged
I-01	Missing on-chain wallet authenticity check in proxy()	Informational	Acknowledged
I-02	Timelock delay has no upper bound	Informational	Fixed

Low Severity Issues

L-01: Single reverting batch in proxy() grieves all other users/wallets in the transaction

Severity: Low	Impact: Low	Likelihood: Low
	Status: Acknowledged	

Description:

The `proxy()` function in `DepositWalletFactory` executes batches sequentially with no error isolation:

```
None
function proxy(Batch[] calldata _batches, bytes[] calldata _signatures) external onlyOperator
{
    require(_batches.length == _signatures.length, ArrayLengthMismatch());
    for (uint256 i; i < _batches.length; ++i) {
        IDepositWallet(_batches[i].wallet).execute(_batches[i], _signatures[i]);
    }
}
```

As per the docs, `proxy()` is used to:

Execute batches on multiple wallets

A malicious user can sign a batch containing a call that passes off-chain simulation but reverts on-chain (e.g., a call to a contract with time-dependent or state-dependent revert logic). Since the operator batches multiple users' transactions into a single `proxy()` call for gas efficiency, one reverting batch takes down the entire transaction.

The operator can mitigate operationally by simulating each batch close to submission time and using smaller batch sizes but cannot fully prevent this at the contract level. State changes

between simulation and on-chain execution (e.g. frontrunning and updating state by the attacker) create a window for griefing.

This is analogous to the bundler griefing problem in ERC-4337 where `innerHandleOp` uses `try/catch` to isolate individual user operations.

Attack path #1 (Frontrun via ownership accept)

1. Transfer ownership, that is step 1 of 2-step transfer process.
2. They send an intent to cancel the ownership transfer.
3. Operator checks everything, cancelling passes in `estimateGas` as it's a legit transaction at this point.
4. They execute it on-chain.
5. Wallet owner frontruns and accepts the ownership which sets the pending owner to `address(0)`.
6. Cancelling reverts as it checks that there is a pending owner which there isn't due to step 5 – this reverts the whole loop and all operations on other wallets also fail.

Attack path #2 (Tight deadline)

This attack path relies on the fact that batch deadline is user-controlled and block timestamps are predictable.

1. Attacker observes how long it takes for a submitted batch to get included on-chain (e.g., always next block).
2. Attacker signs a batch with legitimate calls to allowed targets but sets the deadline to a timestamp that will pass by the next block, e.g., `NEXT_BLOCK_TS - 1`
3. The operator simulates the batch – deadline is still valid, all calls are allowed so validation passes.
4. Transaction gets mined in the next block where `block.timestamp > deadline`.
5. `execute()` reverts with `Expired()` and the entire `proxy()` call fails.

Attack path #3 (Double nonce)

1. User signs two different batches with the same nonce in quick succession.
2. Operator simulates both independently. Since `eth_estimateGas` is stateless, both validations pass.
3. Operator includes both batches in a single `proxy()` call.

4. The first batch executes and nonce gets incremented.
5. The second batch reverts with `InvalidNonce()` which fails the entire `proxy()` call.

Attack path #4 (Signature invalidation via ownership transfer)

1. Attacker controls two wallet addresses – Alice's (wallet owner) and Bob's.
2. Alice signs a batch containing `transferOwnership(Bob)`. Operator relays it as a legitimate operation. Bob is now pending owner.
3. Bob pre-signs an `OwnershipHandover` EIP-712 message.
4. Alice signs any new valid batch and submits it to the operator.
5. Operator simulates – signature is valid against current owner Alice so all calls pass.
6. Bob frontruns the operator and calls `completeOwnershipHandover()` directly (it's a permissionless function and does not need to go through the operator). Ownership is transferred to Bob.
7. `execute()` checks the signature against `owner()` which is now Bob but the batch was signed by Alice so the batch reverts with `InvalidSignature()` and the entire `proxy()` call fails.

Note: the attacker retains full ownership (just as Bob instead of Alice) and can transfer back to Alice and repeat.

The operator may filter some of these off-chain but the deadline vector in particular is hard to mitigate reliably – the safe buffer depends on network conditions, queue depth and batching latency, all of which vary.

Recommendations:

Wrap each `execute()` in a `try/catch` to isolate failures:

```
None
function proxy(Batch[] calldata _batches, bytes[] calldata _signatures) external onlyOperator
{
    require(_batches.length == _signatures.length, ArrayLengthMismatch());
    for (uint256 i; i < _batches.length; ++i) {
        try IDepositWallet(_batches[i].wallet).execute(_batches[i], _signatures[i]) {
            // success
        } catch {
```

```
        emit BatchFailed(_batches[i].wallet, _batches[i].nonce);
    }
}
```

Customer Response:

Acknowledged. The batching capability exists as built-in functionality and is reserved for future use. If and when we decide to enable multi-user batching, we'll revisit the error isolation question at that point.

Fix Review:

Acknowledged.

Informational Severity Issues

I-01: Missing on-chain wallet authenticity check in proxy()

Description:

`proxy()` forwards batch execution to the address specified in `_batches[i].wallet` without on-chain verification that the address was deployed by the factory. While the Polymarket team maintains an off-chain registry of deployed wallets (indexed from `WalletDeployed` events) and validates `batch.wallet` against it before submitting transactions, this check is not enforced at the contract level.

As a defense-in-depth measure, mirroring this validation on-chain would ensure that even in the event of an off-chain misconfiguration, the factory cannot be used to call `execute()` on arbitrary contracts which will pose a security risk.

Recommendations:

Consider adding a `mapping(address => bool) public deployedWallet` set to `true` during `deploy()` and checked in `proxy()`, reinforcing the existing off-chain validation with a lightweight on-chain guard.

Customer Response:

We maintain an off-chain registry of deployed wallets (indexed from `WalletDeployed` events) and validate `batch.wallet` against it before submitting transactions. Doing the additional check will add gas since this is a frequently called function.

I-02: Timelock delay has no upper bound

Description:

`_setTimelockDelay()` does not enforce an upper bound on the `timelockDelay` value. If set to an excessively large value, the `onlyPaused` modifier's check `block.timestamp > paused() + timelockDelay` would never pass, making `unpause()`, `withdrawERC20()`, `withdrawERC1155()`, `revokeAllowance()`, `revokeApprovalForAll()` and `revokeSessionSignerEmergency()` permanently uncalleable on any paused wallet. A reasonable cap ensures that even in the event of an operational mistake or admin key compromise, the impact on user funds is bounded.

Note: The protocol documentation acknowledges the absence of bounds validation on the timelock delay. We include this finding as a defense-in-depth recommendation to bound the impact of a potential key compromise or operational error.

Recommendations:

Introduce a `MAX_TIMELOCK_DELAY` constant and enforce it in `_setTimelockDelay()`:

```
None
uint256 public constant MAX_TIMELOCK_DELAY = 30 days;

function _setTimelockDelay(uint256 _timelockDelay) internal {
    require(_timelockDelay <= MAX_TIMELOCK_DELAY, TimelockDelayTooLarge());
    timelockDelay = _timelockDelay;
    emit TimelockDelaySet(_timelockDelay);
}
```

Customer Response:

Fixed in [817261b](#).

Fix Review:

Fix looks good.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.